Date: __/__/____

Assignment - 5

Handwritten
Creative Memory....

Page No.: ____

Problem Statement - Dept. of Comp. Engineering has student's club named 'Pinnacle Club'. Students of second, third & final year of dept. can be granted membership on request. Similarly, one may cancel it. First node is reserved for president of club & last node is reserved for secretary of club. Write a C program to maintain club member's info using SLL. Store student PRN & Name. Write func. to

a) Add and delete the members
b) Compute total no. of members of club.
c) Display members.
d) Sorting of two linked list
e) Merging of two linked list
f) Reversing using pointers

Objectives -

1) To understand the concept of a singly linked list & dynamic memory allocation using pointers in C.
2) To perform insertion, deletion, display, count nodes, sort, reverse operations on a SLL
3) To develop a menu-driven application for workshop enrollment management

Platform -

64-bit Open source Linux or its derivatives
Open Source C Programming tool like gcc/
Eclipse Editor

Pseudo Code

- Insertion at beginning
  Create new node
  new Node.next = head
  head = newNode

- Deletion by PRN
  Find the node
  Update previous node's next pointer
  Free the deleted node

- Display
  temp = head
  while != NULL
    Print data
    temp = temp.next

- Sorting (Bubble Sort)
  Compare adjacent nodes
  Swap if PRN is greater
  Repeat until sorted

- Reverse
  prev = NULL
  curr = head
  while curr != NULL
    next = curr.next
    curr.next = prev
    prev = curr
  head = prev

- Merge
Go to the last node of list 1
Connect it to head of list 2

Time Complexity

Insert at beggining - O(1)
Insert at end - O(n)
Delete - O(n)
Display - O(n)
Count - O(n)
Search - O(n)
Bubble Sort - $O(n^2)$
Reverse - O(n)
Merge - O(n+m)

Conclusion - In this practical, a Student Club enrollment management system was developed using SLL. The program demonstrates dynamic memory allocation & efficient manipulation of linked data structures. Operations such as insertion, deletion, traversal, counting, sorting & reversing were successfully implemented. This practical enhances the understanding of linked list operations & their applications in real-world record management systems.

1) Why is a linked list preferred over an in this application? What information is stored in each node of circular linked li
A linked list can grow or shrink dynam
Each node stores the student's PRN, nam & pointer to the next node.

2) What is a singly linked list?
A singly linked list is a collection of n where each node contains data & a poi to the next node.

3) What information is stored in each node?
Each node stores the student's PRN, nam & the address of next node.

4) What is the diff. between SLL & arra
An array stores elements in continuous memory locations & has a fixed size while a linked list stores elements in seperate nodes & can grow or shrink

5) What is dynamic memory allocation?
Dynamic memory allocation allows memo to be allocated during program executi using func. like malloc().

Handwritten
Creative Memory....

Date : __/__/____

Page No.: ____

6) Which func. is used to release allocated memory?

The free() func. releases dynamically allocated memory.

7) What is the time complexity of inserting a node at the beginning?

The time complexity is O(1) because only the head pointer needs to be updated.

8) What is the time complexity of searching for a student?

The time complexity is O(n).

9) How is linked list reversed?

A linked list is reversed using three pointers: prev, current & next which changes the direction of links.

10) Which sorting technique can be used in a linked list?

Bubble sort can be used to arrange linked-list elements in order.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    int prn;
    char name[50];
    struct Node *next;
};

struct Node *head = NULL;

struct Node* createNode(int prn, char name[]) {
    struct Node *newNode = (struct
Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->prn = prn;
    strcpy(newNode->name, name);
    newNode->next = NULL;

    return newNode;
}

void insertBeginning(struct Node **head, int prn,
char name[]) {
    struct Node *newNode = createNode(prn,
name);
```

```
    newNode->next = *head;
    *head = newNode;
    printf("Member added successfully.\n");
}

void insertEnd(struct Node **head, int prn, char
name[]) {
    struct Node *newNode = createNode(prn,
name);

    if (*head == NULL) {
        *head = newNode;
    } else {
        struct Node *temp = *head;

        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newNode;
    }

    printf("Member added successfully.\n");
}

void deleteMember(struct Node **head, int prn) {
    struct Node *temp = *head, *prev = NULL;

    while (temp != NULL && temp->prn != prn) {
        prev = temp;
        temp = temp->next;
    }
```

```
    if (temp == NULL) {
        printf("Student not found.\n");
        return;
    }

    if (prev == NULL)
        *head = temp->next;
    else
        prev->next = temp->next;

    free(temp);
    printf("Member deleted successfully.\n");
}

void display(struct Node *head) {
    if (head == NULL) {
        printf("No students registered.\n");
        return;
    }

    printf("\nPRN\tName\n");

    while (head != NULL) {
        printf("%d\t%s\n", head->prn, head->name);
        head = head->next;
    }
}

void countMembers(struct Node *head) {
    int count = 0;
```

```
    while (head != NULL) {
        count++;
        head = head->next;
    }

    printf("Total members: %d\n", count);
}

void sortList(struct Node *head) {
    if (head == NULL)
        return;

    struct Node *i, *j;
    int tempPRN;
    char tempName[50];

    for (i = head; i->next != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->prn > j->prn) {
                tempPRN = i->prn;
                i->prn = j->prn;
                j->prn = tempPRN;

                strcpy(tempName, i->name);
                strcpy(i->name, j->name);
                strcpy(j->name, tempName);
            }
        }
    }
```

```
    printf("List sorted by PRN.\n");
}

void reverseList(struct Node **head) {
    struct Node *prev = NULL;
    struct Node *current = *head;
    struct Node *next = NULL;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }

    *head = prev;
    printf("List reversed successfully.\n");
}

void mergeLists(struct Node **head) {
    struct Node *second = NULL;
    int n, prn, i;
    char name[50];

    printf("Enter number of members in second list:
");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter PRN and Name: ");
        scanf("%d %49s", &prn, name);
        insertEnd(&second, prn, name);
    }
```

```
    if (second == NULL) {
        printf("Second list is empty.\n");
        return;
    }

    if (*head == NULL) {
        *head = second;
    } else {
        struct Node *temp = *head;

        while (temp->next != NULL)
            temp = temp->next;

        temp->next = second;
    }

    printf("Lists merged successfully.\n");
}

int main() {
    int choice, prn;
    char name[50];

    do {
        printf("\n--- PINNACLE CLUB MANAGEMENT
---\n");
        printf("1. Add President\n");
        printf("2. Add Regular Member\n");
        printf("3. Add Secretary\n");
        printf("4. Delete Member by PRN\n");
        printf("5. Display Members\n");
        printf("6. Count Members\n");
        printf("7. Sort Members by PRN\n");
```

```
        printf("8. Merge Two Lists\n");
        printf("9. Reverse List\n");
        printf("0. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter President PRN and Name: ");
                scanf("%d %49s", &prn, name);
                insertBeginning(&head, prn, name);
                break;

            case 2:
                printf("Enter Member PRN and Name: ");
                scanf("%d %49s", &prn, name);
                insertEnd(&head, prn, name);
                break;

            case 3:
                printf("Enter Secretary PRN and Name:
");
                scanf("%d %49s", &prn, name);
                insertEnd(&head, prn, name);
                break;

            case 4:
                printf("Enter PRN to delete: ");
                scanf("%d", &prn);
                deleteMember(&head, prn);
                break;
```

```
            case 5:
                display(head);
                break;

            case 6:
                countMembers(head);
                break;

            case 7:
                sortList(head);
                break;

            case 8:
                mergeLists(&head);
                break;

            case 9:
                reverseList(&head);
                break;

            case 0:
                printf("Exiting program.\n");
                break;

            default:
                printf("Invalid choice!\n");
        }
    } while (choice != 0);

    while (head != NULL) {
        struct Node *temp = head;
        head = head->next;
        free(temp);
    }
```